

Anteproyecto de Trabajo Final
Firmware para un Decoder MPEG-2 prototipado
en FPGA

Esteban Adriel Bustamante

DNI: 436124923

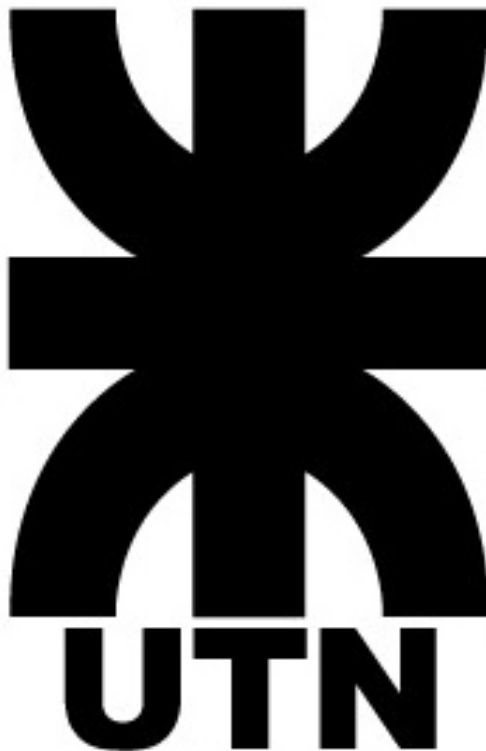
Legajo: 30-6567

Profesor Titular: Ing. Pedro Chahín Rearte

Profesor: Ing. Oscar Almonacid

Tutor: Ing. Ricardo Maldonado

2025



UTN FACULTAD REGIONAL LA RIOJA

Índice

Índice	2
1. Diagnóstico	3
1.1. ¿Qué?	3
1.2. ¿Por qué?	4
1.3. ¿Dónde?	4
1.4. ¿Cuándo?	4
1.5. ¿Quién?	4
2. Objetivos	4
2.1. Objetivo General	4
2.2. Objetivos Específicos	5
3. Áreas del plan de estudio aplicadas	5
4. Justificación	5
5. Antecedentes	6
6. Metodología	6
7. Factibilidad	7
7.1. Recursos materiales	7
7.2. Recursos económicos	7
8. Etapas del proyecto	8
Referencias	10
Índice de figuras	10

1. Diagnóstico

Se comienza por la descripción del problema haciendo uso del método de las 5W, para que de una forma clara podamos esclarecer cuál es la problemática a solucionar y en este caso, la necesidad a suplir.

1.1. ¿Qué?

El problema que se presenta en cuestión es el desarrollo de un firmware para un Decodificador MPEG-2 VLSI prototipado en FPGA, conforme con la norma ISO/IEC 13818-4, con un submuestreo 4:2:0. En la Figura1 observamos el bloque completo. Es importante notar que la entrada es un bitstream en formato MPEG-2 y la salida es RGB, es decir que se podría conectar la salida a un conector HDMI y visualizar un video entrante en alguna pantalla.

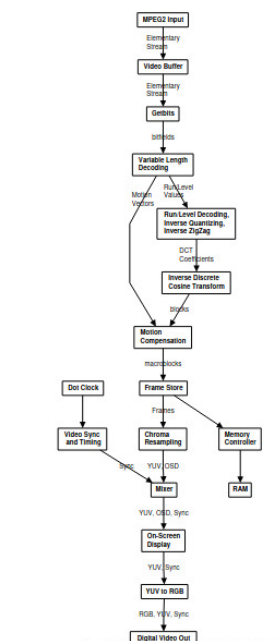


Figura 1: Pipeline del Decoder MPEG-2

1.2. ¿Por qué?

El decodificador en sí no contiene en su arquitectura lazos de control ni tampoco tiene un microprocesador embebido que lo configure para su funcionamiento. Resulta crítico el desarrollo de un firmware robusto que permita controlar en tiempo real el bitstream del decodificador, y por supuesto que se encargue de la inicialización correcta del mismo, ya que en cualquier sistema que se piense para su integración el tráfico de bits no debe cortarse o en todo caso si se interrumpe su normal funcionamiento debiera claramente saberse el causante del error.

1.3. ¿Dónde?

En todos los sistemas donde se utilizan aceleradores por hardware de sistemas de visión artificial, especialmente en este caso VLSI aplicable a dispositivos de bajo consumo de potencia como aquellos que funcionan a batería. Por ej, podríamos tener un Codec MPEG-2 por hardware en un Smartphone que pretenda tener reproducción de videos a una alta tasa de cuadros por segundo. Otra aplicación típica y actual es en la televisión digital DVB, donde los receptores poseen decodificadores MPEG-2, y quien logra la mayor simplicidad cumpliendo la normativa establecida, logra acceder al mercado mas eficientemente.

1.4. ¿Cuándo?

Históricamente, en las primeras épocas los gráficos fueron una cuestión manejada principalmente por software, con sistemas muy básicos con capacidades muy limitadas destinadas a mostrar pixeles en los monitores de las primeras computadoras. A medida que pasaron los años, comenzó la transición con la aparición de las GPU (*Graphical Processing Unit*), que optaron por un punto medio, donde si bien el hardware ya era mucho más específico, los algoritmos en sí seguían corriendo todos por software. Hasta llegar a nuestros días, donde ciertas aplicaciones críticas en velocidad y en consumo de potencia como los mencionados en 1.3 precisan que algoritmos y ciertos tipos de codificaciones corran por hardware.

1.5. ¿Quién?

Un ejemplo de empresa que podría requerir un controlador de gráficos como este sería el caso de la organización openMSP430, que sería la versión open source de la familia de microcontroladores de Texas Instruments. En este caso, ya ha sido realizado el openGFX430, que consiste en un controlador que entre sus bloques posee un decodificador MPEG-2. En nuestro caso, recordemos que el repositorio del bloque MPEG-2 puede encontrarse en la página <https://opencores.org/>.

2. Objetivos

2.1. Objetivo General

Implementar el firmware de nivel industrial de un Decoder de Video MPEG-2 conforme a la norma ISO-IEC 23002-1 prototipado en FPGA, cumplimentando

las etapas de investigación, diseño, implementación y optimización.

2.2. Objetivos Específicos

- Investigar sobre la evolución de algoritmos de compresión de video, particularmente del grupo MPEG (*Moving Pictures Experts Group*)
- Investigar sobre arquitecturas VLSI de dichos algoritmos, particularmente del MPEG-2 en cuestión.
- Investigar y comprender la norma ISO-IEC, sus bases y consecuencias en el desarrollo de un producto.
- Diseñar el firmware con sus respectivas tareas de control y configuración del dispositivo.
- Diseñar el sistema operativo que soportará las tareas propias del firmware.
- Implementar el Sistema Operativo diseñado en el SoC estipulado.
- Implementar el firmware diseñado con sus respectivas tareas.
- Optimizar el sistema realizado en término de eficiencia de tareas y robustez.

3. Áreas del plan de estudio aplicadas

Las áreas estrechamente relacionadas con el presente trabajo son las siguientes:

- **Técnicas Digitales:** en sus 3 niveles, al tratarse de una implementación en hardware del algoritmo de decodificación sobre una FPGA, es absolutamente necesario el entendimiento del funcionamiento de los circuitos digitales tanto combinacionales como secuenciales, incluyendo distintos algoritmos para la realización de diversas operaciones como suma, multiplicación y ni hablar del uso de registros que será gran parte de nuestro trabajo .
- **Sistemas de control:** debido a que la realización del firmware requerirá el diseño e implementación de lazos de control en software, es completamente el entendimiento de las especificaciones de los mismos, particularmente de la materia Sistemas de Control Aplicado, donde se imparte conocimiento sobre controladores digitales.
- **Procesamiento Digital de Imágenes:** ésta área del conocimiento permitió el conocimiento del algoritmo en cuestión, donde fue estudiada toda la cadena de procesamiento de un video para una transmisión de TV Digital Terrestre, entre otros conocimientos que permiten la realización del trabajo actual.

4. Justificación

A partir de la implementación del firmware para este decoder, se podrá configurar, controlar y monitorear el dispositivo implementado en hardware en una arquitectura VLSI. El impacto que busca el mismo es poder generar una flexibilidad a la hora de configurar el dispositivo, abstrayendo al cliente (usuario) de la estructura del hardware, creando una API para poder acceder de manera sencilla al hardware y de esta forma no solo inicializarlo sino también controlarlo y monitorear las diversas tareas y modos de funcionamiento.

5. Antecedentes

Como antecedentes a este proyecto, podemos encontrar diversos esfuerzos tanto de carácter académicos como industriales, haciendo mención especial del trabajo destacado de Claire Billaud (2004) "*MPEG-2 sur architecture mixte processeur + FPGA*" que se traduce a "MPEG-2 sobre una arquitectura mixta procesador + FPGA". Este estudio apunta a implementar un decodificador MPEG-2 sobre una arquitectura mixta de un microprocesador y un FPGA. Se trata de separar la implementación en hardware y en software, por lo que parte del algoritmo se procesa en software y parte en hardware, como es típico en un flujo de diseño de sistema para punto fijo (Ver Figura 2). Este trabajo representa uno de los primeros esfuerzos en implementar algoritmos de compresión en hardware, aun así vale decir que solo parte del algoritmo es implementado en hardware (FPGA al igual que el presente trabajo) mientras que otra parte es implementada en software.

En segundo lugar, aclaramos aquí el trabajo que sirve como punto de partida para este, que a su vez tuvo como punto de partida al anteriormente mencionado. Se trata ni mas ni menos de *mpeg2fpga*, un trabajo realizado por Koenraad De Vleschauwer [3] en el 2017 que implementa el algoritmo completo MPEG-2 bajo norma en hardware, prototipándolo en una FPGA Xilinx. Este trabajo representa la base del presente estudio, puesto que esta implementación en hardware será la de referencia para realizar nuestro firmware alrededor del mismo.

6. Metodología

En el diseño típico de un sistema de punto fijo se emplea el flujo observado en la Figura 2. Si bien el sistema ya se encuentra implementado, corresponderá al proyecto actual implementarlo en el SoC PolarFire MPFS095T, corriendo las diferentes pruebas ofrecidas por el diseñador del sistema para verificarlo a nivel comportamental y funcional. Cabe destacar que este algoritmo está totalmente implementado en hardware.

Para la implementación del firmware se utiliza la metodología TDD (*Test Driven Development*), que como lo indican sus siglas se basa en el desarrollo guiado por pruebas, y se encuentra ampliamente difundido en la actualidad en el mundo del software embebido. En la Figura 3 observamos el ciclo de desarrollo típico de esta metodología. Son numerosas las ventajas de esta decisión de diseño, pero la principal es que al ser el primordial objetivo conseguir un firmware de nivel industrial que se destaque por su robustez y flexibilidad, es esencial conseguir un nivel de *coverage* del 100 %.

El framework de TDD elegido para desarrollar el código es *Ceedling*, pensado para desarrollar firmware para embebidos en ANSI C, lo cual es parte del proyecto. Para la administración del proyecto se hará uso de la herramienta *Org Mode + Projectile*, que permitirá controlar el cumplimiento de los plazos establecidos en la Planificación (Ver Sección 8).

Por ultimo recalcar que para la programación de la placa de desarrollo, es decir del FPGA se hará uso del software provisto por el fabricante Libero® , y para la programación del sistema de microprocesadores se hará uso de otro software del fabricante llamado Softconsole® .

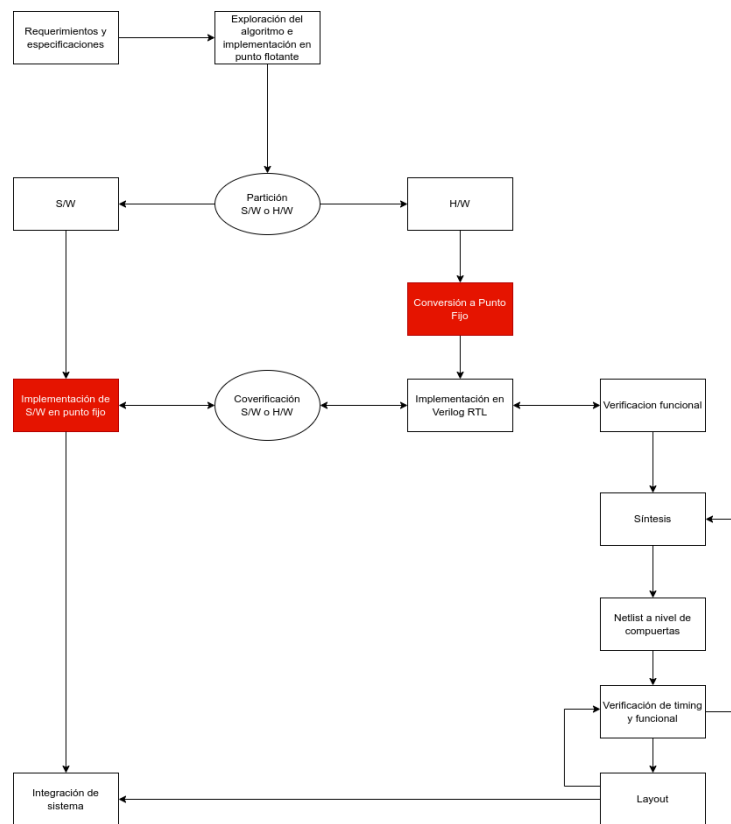


Figura 2: Flujo típico de diseño de sistema en punto fijo

7. Factibilidad

7.1. Recursos materiales

- Computadora
- Placa de desarrollo Polarfire Discovery Kit
- **Software:** Ceedling, ANSI C, Emacs, Org-Mode, SoftConsole®, Libero SoC®.

7.2. Recursos económicos

- Precio de la placa de desarrollo Discovery Kit: 132U\$D

Vale aclarar que el precio de las herramientas de software es cero, debido a que se priorizó la utilización de software libre y código abierto para el desarrollo del mismo, incluyendo las herramientas brindadas por el fabricante de la placa.

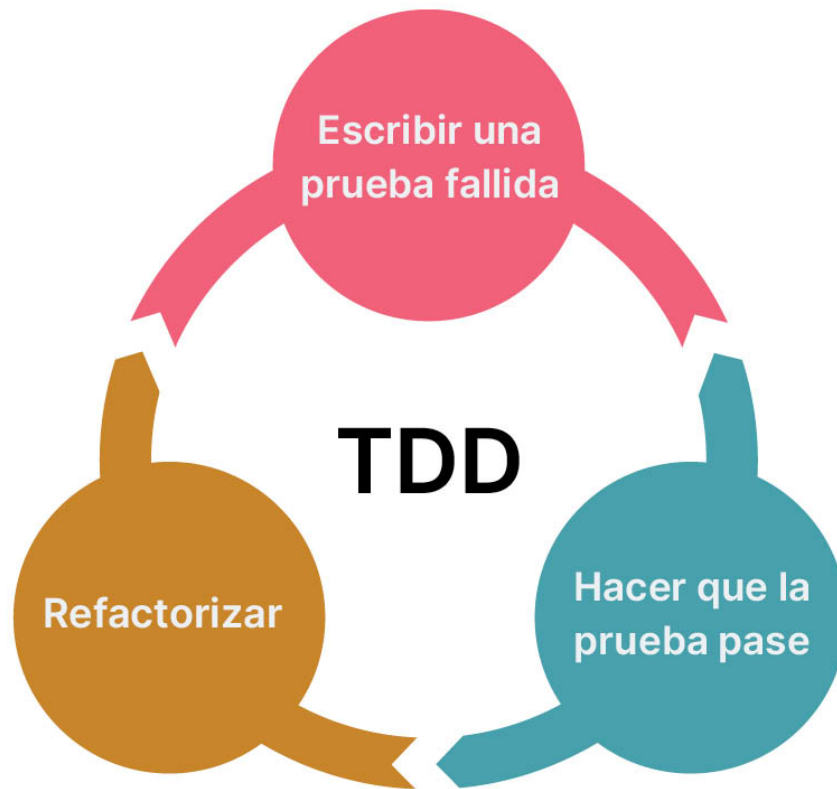


Figura 3: Desarrollo guiado por pruebas

8. Etapas del proyecto

En la figura 4 se muestran las distintas etapas del proyecto a realizar, diferenciando claramente las mismas y las tareas que componen a cada una de ellas.

1. Se da inicio al proyecto.
2. La etapa de investigación consistirá de la búsqueda y lectura de la literatura específica del tema, como lo es [3], con el fin principalmente de poder definir claramente las especificaciones del proyecto.
3. Luego se procede a la lectura y comprensión de la norma vigente para MPEG-2 (ISO/IEC) como se detalla en la bibliografía.
4. Finalmente en esta etapa de investigación incluimos la preparación del flujo de trabajo descrito en la Sección 6.
5. Pasando ya a la etapa de diseño corresponde comenzar con el dimensionamiento y la arquitectura del firmware, es decir proceder con la maqueta

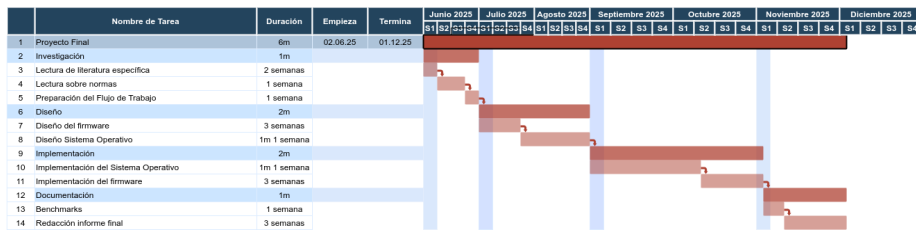


Figura 4: Planificación del proyecto

de las tareas de control y monitoreo, y estructura del archivo de registros de configuración, así como la definición de las interfaces y de la estructura en si de la API.

6. Una vez definido el firmware, resulta mas fácil delimitar el tamaño y funcionalidades del sistema operativo, que a priori por tratarse de un sistema en hardware, se tratará de un sistema operativo en tiempo real. Resulta conveniente realizarlo de esta manera para así no cometer errores de sobre diseño para con el sistema operativo.
7. Pasamos a la etapa de implementación, donde ahora se invierte la lógica pues resulta impráctico implementar el firmware sin un sistema operativo que servirá de plataforma cuando vayamos al chip en la vida real. Aquí entonces se realiza una implementación del sistema operativo en tiempo real para un subsistema de 4 microprocesadores RISC-V.
8. Ahora si procedemos con la implementación de las tareas propias del firmware con sus respectiva documentación técnica.
9. Pasamos a la etapa final de documentación. En primer lugar se ejecutarán distintos benchmarks para poder contrastar no solo la performance de nuestro sistema sino también de cada componente, es decir compararemos nuestro sistema operativo con otros comerciales o de amplia difusión, con un propósito más general.
10. Finalmente se dedicarán 3 semanas a la redacción del informe final que ha de ser presentado para la presentación del Proyecto Final.

Referencias

- [1] C. Billaud. Mise en oeuvre d'un décodeur MPEG-2 sur architecture mixte processeur + FPGA. Master's thesis, ENSEA, 2004.
- [2] R. Çox, F. Kaashoek, and R. Morris. xv6: a simple, unix-like teaching operating system. Agosto 2024.
- [3] K. De Vleeschauwer. MPEG-2 video decoder, Agosto 2017.
- [4] Information technology — MPEG video technologies — Part 7: Versatile supplemental enhancement information messages for coded video bitstreams. Standard, International Organization for Standardization, Geneva, CH, Mar. 2024.
- [5] P. Pessolani. Minix4rt: A real-time operating system based on minix. Master's thesis, Universidad Nacional de La Plata, 2006.
- [6] J. Watkinson. *The MPEG handbook: MPEG-1, MPEG-2, MPEG-4*. Focal Press, Taylor & Francis Group, 2013.

Índice de figuras

1.	Pipeline del Decoder MPEG-2	3
2.	Flujo típico de diseño de sistema en punto fijo	7
3.	Desarrollo guiado por pruebas	8
4.	Planificación del proyecto	9